Pimpri Chinchwad Education Trust's Pimpri Chinchwad College of Engineering
(An Autonomous Institute Affiliated to SPPU, Pune)

DevOps - Kubernetes & Ansible Laboratory Assignment 8

Student Name: Rutuja Jagtap PRN: 123B1B146 Batch: C1
Department: Computer Engineering Academic Year: 2026-2027 Semester: II

Explore Kubernetes Objects and Services and Demonstrate Configuration Management Using Ansible

Kubernetes Core Objects, Service Networking (ClusterIP, NodePort, LoadBalancer, ExternalName), Storage Volumes, Ansible Architecture, Playbook Automation, and End-to-End DevOps Lifecycle

1. Aim

To study important Kubernetes objects and networking services and demonstrate the use of Ansible for automated configuration management.

2. Problem Statement

Managing microservices on clustered infrastructure requires mastery of fundamental Kubernetes objects (Pods, Deployments, Services, Namespaces, ConfigMaps, Secrets, and Persistent Volumes) and network routing types. Concurrently, manual configuration of underlying Linux servers leads to configuration drift, human errors, slow deployment, and maintenance overhead. The task is to create and verify key Kubernetes resources and service types, and resolve the multi-server configuration challenge by implementing an agentless Ansible automation architecture to install, configure, and maintain Nginx web servers across multiple remote nodes using declarative playbooks.

3. Learning Objectives

Area	Learning Objective
Kubernetes Objects	Understand the function and declarative manifest structure of Pods, Deployments, Namespaces, ConfigMaps, Secrets, and Persistent Volumes.
Service Networking	Differentiate between ClusterIP, NodePort, LoadBalancer, and ExternalName service types.
Storage Abstraction	Analyze the PersistentVolume (PV) -> PersistentVolumeClaim (PVC) -> Pod storage binding lifecycle.
Ansible Architecture	Study agentless configuration management via SSH, control nodes, and INI inventory management.
Playbook Automation	Develop and execute an idempotent Ansible playbook automating Nginx installation, service management, and content delivery.
End-to-End DevOps Pipeline	Synthesize how GitHub, Jenkins, Docker, Kubernetes, Helm, and Ansible integrate into a unified CI/CD delivery chain.

4. Kubernetes and Ansible Architectural Overview

Modern enterprise DevOps operates at two complementary tiers: containerized application orchestration and infrastructure configuration management. Kubernetes acts as the container orchestration platform, abstracting compute hardware into logical objects (Pods, Deployments, Services) to provide self-healing, automated scaling, and internal traffic routing.

However, the underlying physical nodes, virtual machines, cloud instances, and external services must still be consistently provisioned, hardened, and maintained.

Ansible provides agentless, idempotent configuration management for physical and virtual machines. Unlike legacy configuration tools that require proprietary client agents and open communication daemons, Ansible operates entirely over standard OpenSSH using Python. By combining Kubernetes for container lifecycle management with Ansible for server provisioning and baseline hardening, organizations achieve a complete, repeatable Infrastructure as Code (IaC) operational framework.

5. Core Concepts, Service Types, and Operational Architecture

The practical explores two major technical domains: Part A investigates Kubernetes object hierarchies and service routing, while Part B explores Ansible control-node automation.

5.1 Architectural Topology

Figure 1 illustrates the dual architecture: on the left, Kubernetes organizes workloads inside isolated Namespaces using Deployments and Services backed by persistent storage; on the right, the Ansible Control Node orchestrates multiple managed web servers via SSH.

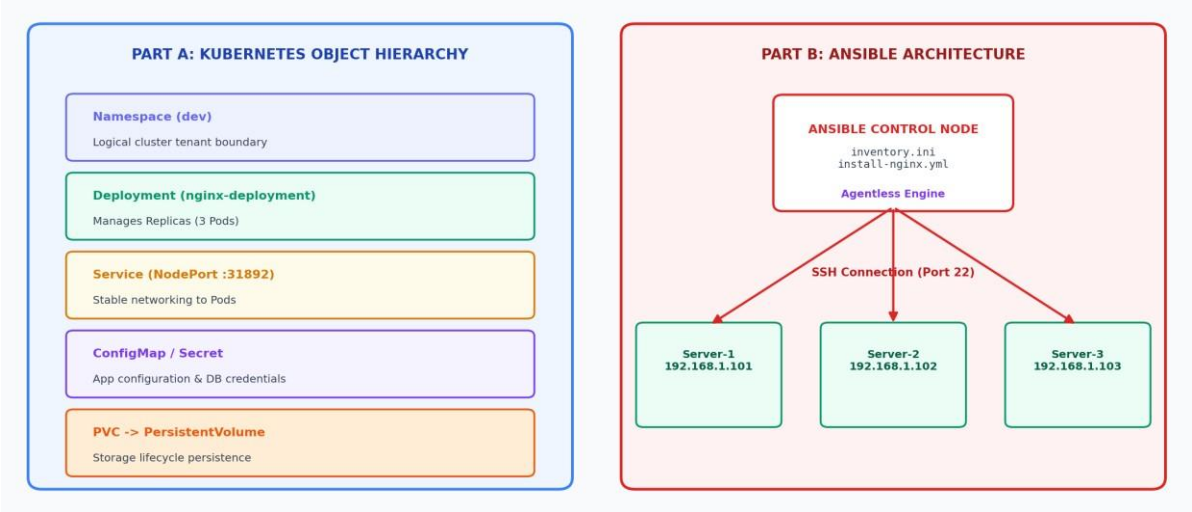


Figure 1. Architectural overview of Kubernetes object hierarchy and Ansible agentless management

5.2 Kubernetes Objects Overview

Object	Role in Kubernetes	Operational Description
Pod	Smallest Computing Unit	Encapsulates one or more closely tied containers sharing network namespace and storage volumes.
Deployment	Declarative Controller	Manages ReplicaSets to provide declarative updates, scaling, and rolling upgrades for Pods.
Service	Networking Abstraction	Defines a logical set of Pods and a stable access policy (IP address and port) across dynamic pod lifecycles.
Namespace	Virtual Cluster Partition	Provides logical separation, scope for unique object names, and resource quota boundaries.
ConfigMap	Configuration Decoupling	Stores non-sensitive key-value configuration pairs injected into Pods as environment variables or volume mounts.
Secret	Sensitive Data Store	Stores sensitive data (passwords, tokens, keys) in base64 format, preventing credential leaks in manifests.
Persistent Volume (PV)	Cluster Storage Asset	Provisions physical cluster storage independent of the ephemeral lifecycle of individual Pods.

5.3 Kubernetes Service Types Comparison

Kubernetes provides four distinct Service types to route networ affic to Pods:

Service Type	Accessibility Scope	Primary Use Case & Routing Mechanism
ClusterIP (Default)	Internal Only	Exposes the Service on a cluster-internal IP. Used for backend communication between microservices.
Service Type	**Accessibility Scope**	**Primary Use Case & Routing Mechanism**
NodePort	External via Node Port	Allocates a high port (30000-32767) on every cluster node, routing traffic from <Node-IP>:<NodePort> to Pods.
LoadBalancer	Cloud External Public IP	Provisions a cloud provider load balancer (AWS NLB, GCP LB) that forwards external traffic into the NodePort.
ExternalName	External DNS Alias	Maps the Service directly to an external DNS CNAME record (e.g., example.com) without proxying.

5.4 Storage Workflow: PV, PVC, and Pods

Kubernetes decouples storage administration from developer consumption: an administrator provisions a PersistentVolume (PV), an application author claims capacity via a PersistentVolumeClaim (PVC), and the Pod mounts the PVC as a volume.

- PersistentVolume (PV): The actual storage piece in the cluster (NFS, local SSD, cloud block storage).
- PersistentVolumeClaim (PVC): A request for storage with specific size and access modes (ReadWriteOnce, ReadWriteMany).
- Pod Mounting: Pod references the PVC in its volumes definition, mounting durable storage to target container paths.

5.5 End-to-End DevOps Lifecycle Pipeline

The combination of Git, Jenkins, Docker, Kubernetes, Helm, and Ansible forms an enterprise CI/CD automation pipeline:

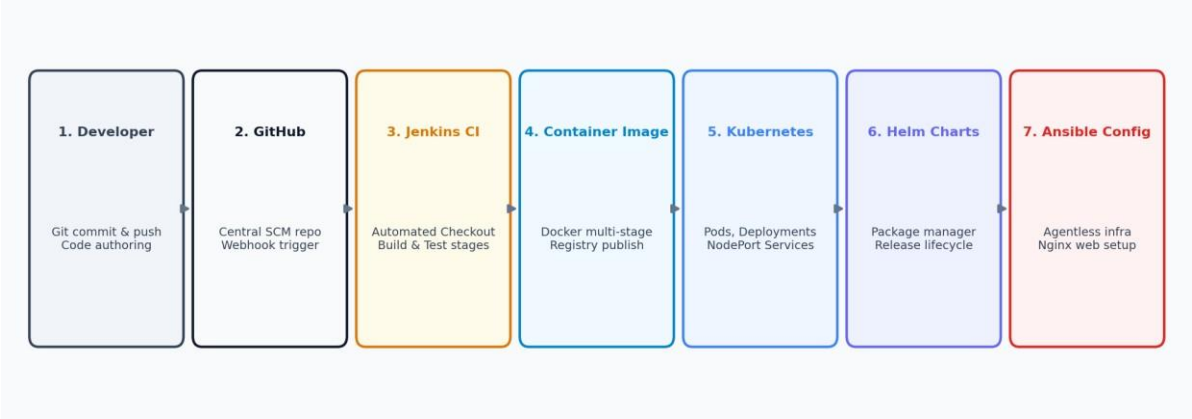


Figure 2. End-to-end DevOps automated software delivery and infrastructure lifecycle

6. Installation, Prerequisites, and Environment Configuration

The practical requires an active Kubernetes cluster (Minikube / k8s) with kubectl, as well as an Ansible Control Node running on Linux with OpenSSH. **6.1 Prerequisites Matrix**

Component	Requirement	Verification Method
Kubernetes Minikube	v1.28.0+ running cluster	kubectl cluster-info & kubectl get nodes
kubectl CLI	Client configured with admin context	kubectl version --client

Ansible Control Node	Python 3.10+ & ansible-core 2.16+	ansible --version
Managed Linux Nodes	Ubuntu/Debian nodes with Python & SSH	ansible all -i inventory.ini -m ping

6.2 Ansible Project Directo ry Structure

File / Directory	Role in Ansible Project	Description
inventory.ini	Host Inventory	Defines managed server IP addresses grouped under [webservers].

File / Directory	Role in Ansible Project	Description
install-nginx.yml	Ansible Playbook	YAML file defining tasks: apt install, service start, and HTML copy.
ansible.cfg	Ansible Settings	Configures inventory path, remote user, and host key checking rules.

7. Kubernetes Core Objects Implementation (Part A)

The core Kubernetes objects were declared using YAML manifests and applied via kubectl:

7.1 Pod Manifest (pod.yaml) **pod.yaml**

```
apiVersion: v1
kind: Pod metadata:
  name: nginx-pod
labels:
    app: nginx spec:
containers:     - name:
nginx       image:
nginx:latest       ports:
        - containerPort: 80
```

7.2 Deployment Manifest (deployment.yaml)

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:   name: nginx-
deployment spec:
replicas: 3   selector:
matchLabels:       app:
nginx   template:
metadata:       labels:
        app: nginx     spec:
containers:          - name:
nginx           image:
nginx:latest           ports:
            - containerPort: 80
```

7.3 Namespace, ConfigMap, and Secret Creation

Creating namespaces, configuration maps, and credentials using imperative CLI commands:

Namespace, ConfigMap & Secret Commands

```
kubectl create namespace dev
kubectl create configmap app-config --from-literal=APP_ENV=production
kubectl create secret generic db-secret --from-literal=username=admin --
from-literal=password=admin123
```

8. Ansible Configuration Management Case Study (Part B)

Case Study: An enterprise organization requires identical, automated provisioning of Nginx web servers across multiple remote Linux servers. Manual installation is error-prone, inconsistent, and unscalable. Ansible automates this workflow from a single control node.

8.1 Inventory Configuration (inventory.ini)

inventory.ini

```
[webservers]
192.168.1.101 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_rsa
192.168.1.102 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_rsa
```

8.2 Playbook Specification (install-nginx.yml)

install-nginx.yml

```
---
-       name: Configure Web
Servers   hosts: webservers
become: yes   tasks:
-       name: Install Nginx
apt:         name: nginx
state: present
update_cache: yes

-       name: Start Nginx
Service       service:
        name: nginx
state: started
enabled: yes

-       name: Deploy Custom Web
Page       copy:
content: |
          <h1>Web Server Configured Using Ansible</h1>
dest: /var/www/html/index.html
```

9. Service Networking and Ansible Idempotency Execution

1. Kubernetes Service Expose: The nginx-deployment was exposed via NodePort on port 80:

Kubernetes Service Exposure

```
kubectl expose deployment nginx-deployment --type=NodePort --port=80 kubectl
get svc -o wide
```

2. Ansible Ad-Hoc & Playbook Execution: Testing connectivity and executing the playbook:

Ansible Execution & Web Verification

```
ansible all -i inventory.ini -m ping
ansible-playbook -i inventory.ini install-nginx.yml curl
http://192.168.1.101
```

3. Idempotency Verification: Re-running ansible-playbook confirmed zero changes (changed=0, ok=4), proving that Ansible only applies modifications when the actual state differs from the desired state.

10. Execution Workflow

The step-by-step practical sequence executed across both technical domains:

Step #	Domain	Command Executed	Operational Purpose
Step #	Domain	Command Executed	Operational Purpose
1	Kubernetes	kubectl apply -f pod.yaml	Creates single isolated nginx-pod.
2	Kubernetes	kubectl apply -f deployment.yaml	Creates nginx-deployment with 3 pod replicas.
3	Kubernetes	kubectl expose deployment --type=NodePort	Exposes deployment externally via NodePort on port 80.
4	Kubernetes	kubectl create namespace dev	Creates logical tenant partition namespace 'dev'.

5	Kubernetes	kubectl create configmap & secret	Creates app-config and db-secret key-value stores.
6	Ansible	ansible --version	Verifies Ansible core engine and Python 3 runtime.
7	Ansible	ansible all -i inventory.ini -m ping	Validates SSH connectivity to all webservers (pong).
8	Ansible	ansible-playbook -i inventory.ini install-nginx.yml	Installs Nginx, starts daemon, and deploys index.html.
9	Ansible	curl http://<server-ip>	Asserts live web page response from managed servers.
10	Ansible	Re-run ansible-playbook	Verifies configuration idempotency (changed=0).

11. Security, Best Practices and Good Practices

Practice	DevOps Implementation & Application
Secret Encryption	Never commit plaintext passwords in Kubernetes manifests or Ansible playbooks; utilize Kubernetes Secrets and Ansible Vault.
Least Privilege Roles	Enforce Kubernetes RBAC and restrict Ansible SSH execution to dedicated non-root users with sudo privileges.
Idempotent Playbooks	Always design Ansible tasks using declarative modules (apt, service, copy) instead of raw shell commands to preserve idempotency.
Health Probes	Configure liveness and readiness probes in Kubernetes Pods to ensure Services route traffic only to healthy containers.
Host Key Verification	Enforce strict host key checking in production ansible.cfg to prevent man-in-the-middle attacks.
Storage Reclaim Policies	Configure PersistentVolume reclaim policies (Retain vs Delete) carefully to protect stateful data from accidental deletion.

12. Practical Verificatio ı Checklist

Activity	Verification Point	Evidence to Attach
K8s Objects Setup	Pod, Deployment, Namespace, ConfigMap, and Secret created.	CLI get pods, deployments, secrets screenshot (Evidence A).
K8s Services & Endpoints	NodePort service exposes deployment; curl returns 200 OK.	Service table and endpoint curl screenshot (Evidence B).
Ansible Control Node	Ansible installed; ad-hoc ping returns SUCCESS pong.	Ansible version and ping module screenshot (Evidence C).
Playbook & Idempotence	Playbook finishes with changed=3; curl verifies web page.	Playbook execution and curl response screenshot (Evidence D).

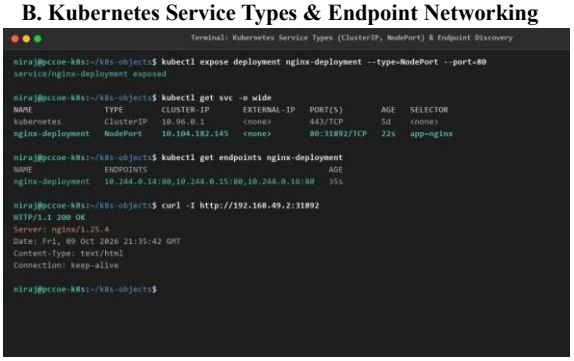
13. Practical Evidence

The terminal captures below record the complete practical execution covering Kubernetes objects, service networking, Ansible ad-hoc connectivity, and playbook web server automation:

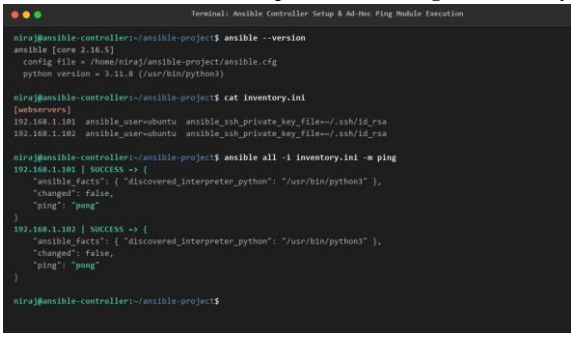
A. Kubernetes Objects Verification (Pods, Deployments, Secrets)



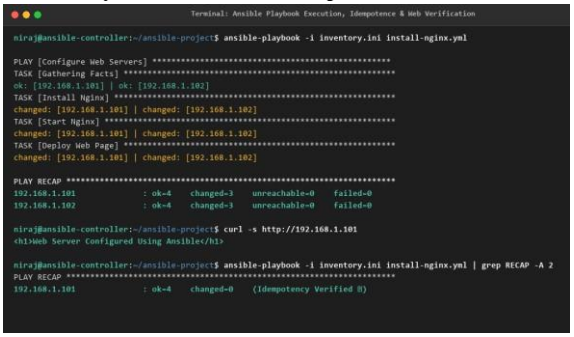
B. Kubernetes Service Types & Endpoint Networking



C. Ansible Control Node Setup & Ad-Hoc Ping Connectivity



D. Ansible Playbook Execution, Idempotence & Web Verification



14. Result

The laboratory assignment successfully explored core Kubernetes objects and networking services, and demonstrated automated configuration management using Ansible. In Part A, an isolated Pod, a 3-replica Deployment, a development Namespace, a ConfigMap, a Secret, and a NodePort Service were created and verified. In Part B, an agentless Ansible architecture was deployed; the inventory was validated using ad-hoc ping modules, and an automated playbook successfully installed Nginx, enabled the system service, deployed custom HTML content across managed nodes, and proved configuration idempotence.

15. Conclusion

Integrating Kubernetes container orchestration with Ansible configuration management represents the pinnacle of modern enterprise DevOps engineering. Kubernetes manages dynamic, self-healing microservices through declarative APIs and service discovery, while Ansible guarantees that underlying virtual servers and host environments are provisioned consistently, securely, and without configuration drift. Together with GitHub, Jenkins, Docker, and Helm, these tools form a resilient, automated CI/CD pipeline capable of delivering reliable software at high velocity.